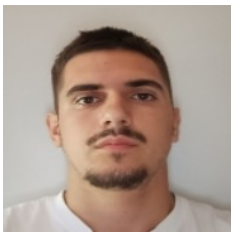


Universidade do Minho
Escola de Engenharia

Trabalho Prático POO

Grupo G036

Gabriel Faria	a111532	
Ricardo Rodrigues	a111225	
Rui Gomes	a111273	

Licenciatura em Engenharia Informática
Universidade do Minho
2025/2026

Conteúdo

1 Introdução **2**

2 Arquitetura do Sistema **2**
2.1 Organização em Packages . . . 2
2.2 Dispositivos . . . 2
2.3 Casa e Divisão . . . 3
2.4 Mecanismos . . . 3
2.5 Utilizador e Histórico de Interações . . . 3
2.6 Motor de Sugestões . . . 3
2.7 Modelo e Serviços . . . 4
2.8 Padrão Model-View-Controller . . . 4
2.9 Diagrama de Classes . . . 4

3 Decisões de Design **5**
3.1 Agregação vs. Composição . . . 5
3.2 DomusControl . . . 6
3.3 Tratamento de Exceções . . . 6

4 Descrição da Aplicação **6**
4.1 Menu Principal . . . 6
4.2 Gestão de Utilizadores . . . 6
4.3 Gestão de Casas . . . 6
4.4 Criação de Automações . . . 6
4.5 Criação de Escalonamentos . . . 6
4.6 Criação de Cenários . . . 7
4.7 Sugestões Inteligentes . . . 7
4.8 Estatísticas . . . 7
4.9 Persistência . . . 7

5 Testes Automatizados **7**

6 Conclusão **7**

1 Introdução

O **DomusControl** é uma aplicação de automação doméstica desenvolvida no âmbito da unidade curricular de Programação Orientada aos Objectos (POO), inspirada em sistemas como o *Home Assistant*. O sistema permite gerir casas, divisões, dispositivos e mecanismos de automatização através de uma interface textual por menus.

O desenvolvimento seguiu rigorosamente os princípios da programação orientada aos objectos, como encapsulamento, herança, polimorfismo e abstração. O relatório está organizado da seguinte forma: a Secção 2 descreve a arquitetura de classes do sistema, a Secção 3 apresenta as principais decisões de design, a Secção 4 descreve a aplicação e as suas funcionalidades, a Secção 5 apresenta os testes automatizados realizados e a Secção 6 conclui o relatório.

2 Arquitetura do Sistema

A arquitetura do DomusControl segue um modelo orientado aos objectos, organizado em packages bem definidos. O sistema é composto por dispositivos domésticos de vários tipos, agrupados em divisões, que por sua vez pertencem a casas. Os mecanismos de automação - automações, escalonamentos e cenários - operam sobre estes dispositivos. O DomusControl coordena todas as operações, delegando em serviços especializados.

2.1 Organização em Packages

O projeto está organizado nos seguintes packages principais:

- `org.domuscontrol` - Classe `Utilizador` e ponto de entrada (`Main`)
- `org.domuscontrol.casa` - Classes `Casa` e `Divisao`
- `org.domuscontrol.dispositivos` - Hierarquia de dispositivos
- `org.domuscontrol.mecanismos` - Automações, escalonamentos, cenários e ações
- `org.domuscontrol.model` - `DomusControl` e serviços
- `org.domuscontrol.model.services` - Serviços especializados do modelo
- `org.domuscontrol.exceptions` - Exceções específicas do domínio
- `org.domuscontrol.ui` - Interface textual (`TextUI`, `Menu`)
- `org.domuscontrol.ui.menus` - Sub-menus por entidade

2.2 Dispositivos

A hierarquia de dispositivos é baseada em herança a partir de uma classe abstrata `Dispositivo`. Esta arquitetura permite adicionar novos tipos de dispositivos sem alterar o código existente.

Classe	**Atributos específicos**
`Dispositivo` (base)	`id`, `marca`, `modelo`, `consumoPorHora`, `estado`
`ArCondicionado`	`temperatura`, `tempMin`, `tempMax`
`Aspirador`	`intensidade`, `intensidadeMax`
`CamaraVigilancia`	`visaoNoturna`, `visaoInfraVermelho`
`Coluna`	`volume`
`Forno`	`temperatura`, `tempMax`
`Lampada`	`intensidade`, `intensidadeMax`
`Persiana`	`percentagemAbertura`
`Portao`	`percentagemAbertura`
`Televisao`	`volume`, `brilho`

O `Dispositivo` tem um contador estático `proxId` para garantir identificadores únicos e auto-incrementados.

2.3 Casa e Divisão

A `Casa` agrega `Divisao` (num `Map<Integer, Divisao>`) e cada `Divisao` agrega `Dispositivo`. A `Casa` também agrega diretamente listas de mecanismos: `List<Automacao>`, `List<Escalonamento>` e `List<Cenario>`.

A relação é de **agregação**: os dispositivos existem independentemente das divisões e as casas existem independentemente dos utilizadores. Não são usados clones ao guardar nas coleções internas.

2.4 Mecanismos

Todos os mecanismos derivam da classe `Mecanismo`, que contém:

- `identificador` (auto-gerado), `nome`, `descricao`
- `List<AcaoDispositivo> acoes` - lista de ações a executar
- Método `executarAcoes(Casa)` que itera e executa todas as ações

As ações implementam a interface `AcaoDispositivo`:

Ação	Efeito
`AcaoLigar`	Liga um dispositivo numa divisão
`AcaoDesligar`	Desliga um dispositivo
`AcaoMudarTemperatura`	Altera a temperatura do A/C ou Forno
`AcaoMudarIntensidadeLampada`	Altera a intensidade de uma lâmpada

As condições de automação implementam a interface `CondicaoAutomacao`:

Condição	Avaliação
`CondicaoDispositivoLigado`	Verdade se o dispositivo estiver ligado
`CondicaoDispositivoDesligado`	Verdade se o dispositivo estiver desligado
`CondicaoTemperaturaMaiorQue`	Verdade se a temp. do A/C exceder o limite
`CondicaoConsumoMaiorQue`	Verdade se o consumo total exceder o limite

Automação: verifica a condição em cada avaliação e executa as ações quando a condição passa de falsa para verdadeira (evitando execuções repetidas). O flag `condicaoAtiva` gere este estado.

Escalonamento: dispara as ações numa hora específica (em minutos desde meia-noite). Pode ser diário ou por dias específicos da semana. O campo `ultimoTempoDisparado` evita múltiplos disparos no mesmo minuto.

Cenário: tem duas listas de ações - `acoes` (ativação) e `acoesDesativacao`. O flag `ativo` controla o estado. Os métodos `ativar()`, `desativar()` e `alternar()` gerem as transições.

2.5 Utilizador e Histórico de Interações

O `Utilizador` mantém duas coleções de casas:

- `Map<Integer, Casa> casas` - casas de acesso normal
- `Map<Integer, Casa> casasAdmin` - casas com permissão de administrador

Regista também um histórico de `List<Interacao>`, onde cada `Interacao` guarda momento, casa, divisão, dispositivo e tipo de ação. Este histórico alimenta o motor de sugestões.

2.6 Motor de Sugestões

O método `gerarSugestoes()` no `Utilizador` analisa o histórico de interações e agrupa-as por *(hora, dispositivo, ação)*. Quando um padrão se repete pelo menos 3 vezes, gera um `Escalonamento` sugerido. O utilizador pode aceitar a sugestão e ela é adicionada a uma casa da sua escolha.

2.7 Modelo e Serviços

A classe `DomusControl` funciona como **fachada** do sistema. Mantém os mapas de utilizadores e casas e delega operações especializadas em serviços:

Serviço	Responsabilidade
`UtilizadoresECasasModel`	Associar/remover utilizadores de casas, permissões
`DispositivoModel`	Adicionar/remover/operar dispositivos
`DivisoesModel`	Gestão de divisões
`AutomacoesModel`	Listagem de condições disponíveis
`EscalonamentosModel`	Serviço de escalonamentos
`CenariosModel`	Serviço de cenários
`FicheirosModel`	Guardar/carregar estado em ficheiro binário
`TopsModel`	Estatísticas e rankings

2.8 Padrão Model-View-Controller

A arquitetura do DomusControl segue o padrão **Model-View-Controller (MVC)**, separando claramente as responsabilidades entre a lógica de negócio, a interface com o utilizador e o controlo do fluxo da aplicação. A figura seguinte ilustra esta organização.

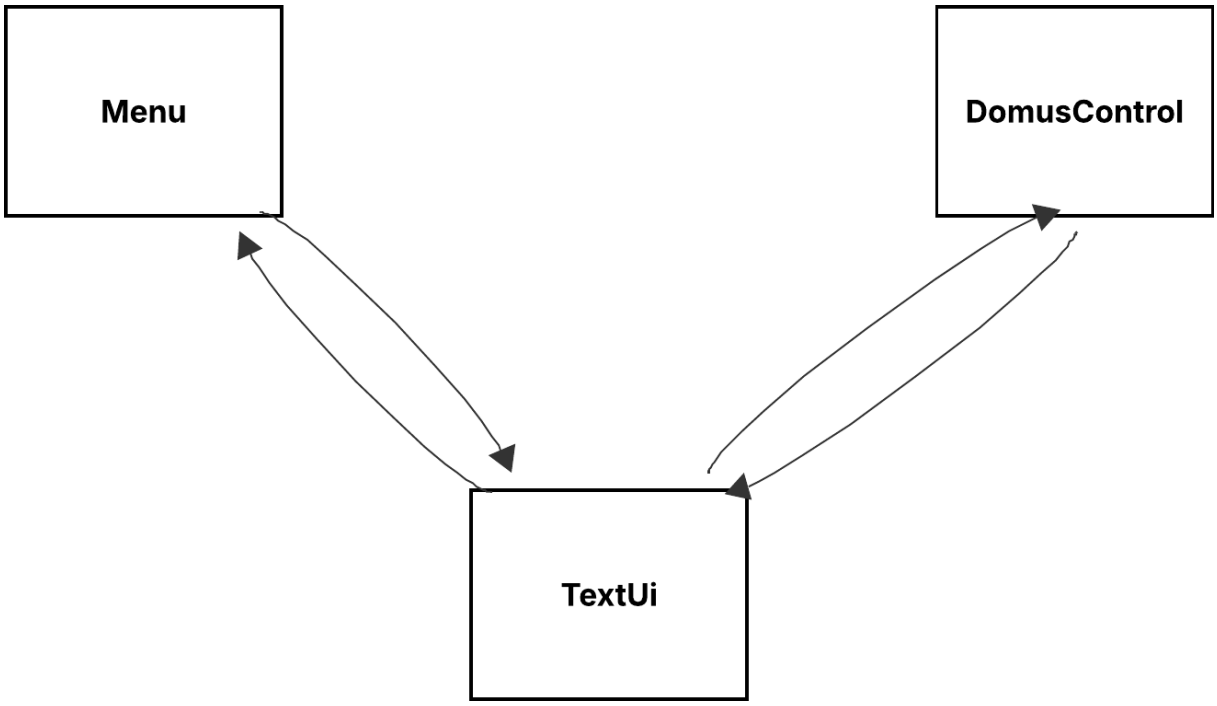


Figura 1: Arquitetura MVC do DomusControl.

2.9 Diagrama de Classes

O diagrama de classes seguinte apresenta as principais classes do sistema e as relações entre elas.



Figura 2: Diagrama de classes do DomusControl.

3 Decisões de Design

3.1 Agregação vs. Composição

Uma das principais decisões foi usar **agregação** em vez de composição. Os mecanismos e as ações são referenciados por referência nas coleções - não são clonados ao ser adicionados. Esta abordagem garante que uma alteração ao objeto original se reflete em quem o detém, e permite partilha entre componentes.

Os *getters* defensivos (que retornam novas listas) continuam a ser usados nas APIs públicas para proteger a coleção interna de modificações externas acidentais.

3.2 DomusControl

O `DomusControl` funciona como fachada: a UI nunca acede diretamente a `Casa` ou `Utilizador` para operações complexas - usa os métodos da fachada ou dos serviços. Isto simplifica a interface e isola a lógica de negócio.

3.3 Tratamento de Exceções

Foram definidas exceções específicas para cada caso de erro do domínio (ex: `CasaInexistenteException`, `DispositivoInexistenteException`).

4 Descrição da Aplicação

4.1 Menu Principal

Ao iniciar, o utilizador vê o menu principal com 6 opções:

```
1 - Gestao de Utilizadores
2 - Gestao de Casas
3 - Estatisticas
4 - Avancar Tempo
5 - Guardar Estado
6 - Carregar Estado
0 - Sair
```

4.2 Gestão de Utilizadores

- **Adicionar/Listar/Remover** utilizadores
- **Associar casa** a utilizador (com papel de admin ou normal)
- **Gerir permissões** (tornar admin / revogar admin)
- **Ver Sugestões de Escalonamentos** - analisa o histórico de interações e propõe escalonamentos automáticos

4.3 Gestão de Casas

O menu de casa adapta-se ao papel do utilizador. Um utilizador normal pode listar divisões, dispositivos, automações, escalonamentos e cenários. Um administrador tem acesso total a menus de gestão:

- **Divisões**: adicionar, listar, remover; e dentro de cada divisão, gerir dispositivos
- **Automações**: criar automações com condição + ações compostas; listar; remover; avaliar
- **Escalonamentos**: criar com hora (HH:MM) e dias da semana; listar; remover
- **Cenários**: criar com ações de ativação e desativação; ativar; desativar; listar; remover

4.4 Criação de Automações

O utilizador escolhe sequencialmente: o nome da automação, o tipo de condição (dispositivo ligado/-desligado, temperatura do A/C, consumo total), os parâmetros da condição (IDs de divisão e dispositivo, limiares) e uma ou mais ações a executar (ligar, desligar, mudar temperatura, mudar intensidade).

Quando avaliadas, as automações verificam a condição face ao estado atual da casa e executam as ações se a condição for satisfeita (e não o era antes).

4.5 Criação de Escalonamentos

O utilizador define o nome do escalonamento, a hora de disparo no formato HH:MM, se dispara todos os dias ou em dias específicos da semana, e as ações a executar (com a mesma estrutura das automações).

Quando o sistema avança no tempo (*Avançar Tempo*), todos os escalonamentos de todas as casas são verificados e disparados se a hora e o dia corresponderem.

4.6 Criação de Cenários

O utilizador define o nome e descrição do cenário, as ações de ativação (ex: desligar luzes, fechar cortinas) e, opcionalmente, as ações de desativação (ex: ligar luzes novamente). Exemplos de cenários típicos: "Sair de Casa", "Ver Cinema", "Jantar com Amigos", "Deitar".

4.7 Sugestões Inteligentes

O sistema regista cada interação do utilizador com os dispositivos. O método `gerarSugestoes()` agrupa as interações por (hora, dispositivo, tipo de ação) e, quando um padrão aparece ≥ 3 vezes, gera um `Escalonamento` sugerido.

Na interface, o utilizador vê as sugestões e pode aceitá-las ou ignorá-las. As aceites são adicionadas à casa selecionada.

4.8 Estatísticas

O módulo de estatísticas fornece:

- Casa com maior consumo energético total
- Três dispositivos mais utilizados (tempo ligado e número de ativações)
- Três divisões com mais dispositivos (e respetiva casa)

4.9 Persistência

O estado completo da aplicação (utilizadores, casas, dispositivos, mecanismos) é serializado para ficheiro binário através de `FicheirosModel`. O utilizador pode guardar e carregar o estado em qualquer momento da execução.

5 Testes Automatizados

O projeto inclui testes unitários organizados em classes de teste, cobrindo todos os módulos principais do sistema.

Módulo	**Classes de teste**
Dispositivos	`DispositivoTest`, `ArCondicionadoTest`, `LampadaTest`, ...
Casa/Divisão	`CasaTest`, `DivisaoTest`
Mecanismos	`AutomacaoTest`, `EscalonamentoTest`, `CenarioTest`
Ações	`AcaoLigarTest`, `AcaoDesligarTest`, `AcaoMudarTemperaturaTest`, ...
Condições	`CondicaoDispositivoLigadoTest`, `CondicaoTemperaturaMaiorQueTest`, ...
Interação	`InteracaoTest`
Utilizador	`UtilizadorTest`
Modelo	`DomusControlTest`

De seguida apresenta-se uma visão gráfica da distribuição dos testes por módulo:

6 Conclusão

O desenvolvimento do DomusControl permitiu consolidar os conceitos fundamentais da Programação Orientada aos Objectos. O projeto cumpre integralmente os quatro patamares de requisitos definidos no enunciado. Ao nível dos requisitos base, foi implementada a gestão completa de utilizadores, casas, divisões e nove tipos de dispositivos, com suporte para operação individual de cada dispositivo. No patamar seguinte, foram desenvolvidas as estatísticas solicitadas - casa com maior consumo, três dispositivos mais utilizados e três divisões com mais dispositivos - bem como a persistência total do estado em ficheiro binário. A secção das automações, escalonamentos e cenários foi também satisfeita, assim como a parte das sugestões automáticas, que analisa o histórico de interações do utilizador e propõe escalonamentos relevantes, que podem ser aceites e integrados na casa escolhida.

A principal decisão arquitetural - o uso de agregação em vez de composição - revelou-se determinante para a coerência do sistema: qualquer alteração a um dispositivo reflete-se imediatamente em todos os componentes que o referenciam, sem necessidade de sincronização manual.

Por fim, os testes unitários, com 100% de sucesso, demonstram a robustez da solução desenvolvida e foram essenciais para validar o comportamento correto.